

Curso: Programación Avanzada

Guía: 10

Título: Persistencia con H2 y Patrón Repository

Duración estimada: 150 minutos

Docente: Christian Andrés Candela

PERSISTENCIA CON H2 Y PATRÓN REPOSITORY

OBJETIVO

En la **Guía 06** definimos las interfaces de repositorio en el dominio e implementamos repositorios en memoria con HashMap . En la **Guía 09** transformamos los casos de uso en servicios Spring, preparándolos para recibir implementaciones reales. Ahora es momento de completar el ciclo:

Reemplazar los repositorios en memoria por implementaciones reales con H2 y Spring Data JPA, sin modificar ni una línea de los casos de uso.

Al finalizar, los estudiantes serán capaces de:

1. Entender **H2** como base de datos embebida para desarrollo
2. Definir **interfaces Repository** en la capa de dominio
3. Implementar repositorios con **Spring Data JPA** en infraestructure
4. Mapear entre **entidades de dominio** y **entidades JPA**
5. Aplicar el principio de **inversión de dependencias** (dominio no depende de JPA) **Principio**

clave: H2 es un detalle de implementación. El dominio solo conoce interfaces Repository. **¿Qué**

cambia y qué NO cambia?

Antes (Guías 06-09): Después (ESTA GUÍA):

Controller → UseCase → SolicitudRepository Controller → UseCase → SolicitudRepository ↓ ↓

SolicitudRepoEnMemoria SolicitudJpaRepository

(HashMap) (H2 + Spring Data JPA)

Observa: El Controller y los Use Cases permanecen **exactamente iguales**. Solo cambia la implementación del repositorio. Este es el poder de la inversión de dependencias.

CONCEPTOS BÁSICOS

H2 Database

Base de datos relacional embebida escrita en Java, que puede ejecutarse en memoria o persistir en archivo. Ideal para desarrollo y testing.

Características:

- Sin instalación ni servidor externo
- Consola web integrada
- Compatible con SQL estándar
- Rápida y ligera

El Problema del Mapeo Objeto-Relacional

En las guías anteriores construimos un dominio rico en Java: objetos con comportamiento, value objects inmutables, agregados con invariantes. Pero las bases de datos relacionales no hablan de objetos — hablan de **tablas, filas y columnas**.

Esto genera un desajuste fundamental conocido como **impedance mismatch** (desajuste de impedancia):

MUNDO DE OBJETOS (Java) MUNDO RELACIONAL (BD)

Solicitud tabla: solicitudes

- └─ CodigoSolicitud (VO) → columna: codigo (VARCHAR)
- └─ Prioridad (VO) → columnas: prioridad_nivel, prioridad_justificacion | └─ nivel (un VO se "aplana" en columnas)
 - └─ justificacion
- └─ List<EventoHistorial> → tabla: eventos_historial (relación 1:N) | └─ (colección) (una lista se convierte en otra tabla) | └─ Usuario responsable → columna: responsable_id (FK)
 - └─ (objeto completo) (un objeto se reduce a un ID)
- └─ EstadoSolicitud (enum) → columna: estado (VARCHAR)
 - └─ (tipo seguro) (un enum se serializa como texto)

Observa los problemas:

1. **Objetos embebidos** → **columnas planas**: Prioridad tiene dos campos pero en la BD son dos columnas sueltas
2. **Colecciones** → **tablas separadas**: List<EventoHistorial> requiere una tabla aparte con foreign key
3. **Referencias a objetos** → **solo IDs**: Usuario responsable se almacena como un número
4. **Herencia y polimorfismo**: no tienen equivalente directo en tablas
5. **Comportamiento**: las tablas no tienen métodos

ORM: Object-Relational Mapping

Un **ORM** (Mapeo Objeto-Relacional) es una técnica que resuelve este desajuste **automatizando la traducción** entre objetos Java y tablas relacionales. En lugar de escribir SQL manualmente para cada operación:

```
// ❌ Sin ORM: escribir SQL manualmente
String sql = "INSERT INTO solicitudes (codigo, descripcion, estado, solicitante_id, " + "canal_origen, prioridad_nivel,
        prioridad_justificacion, fecha_creacion) " + "VALUES (?, ?, ?, ?, ?, ?, ?, ?)";

PreparedStatement stmt = connection.prepareStatement(sql);
stmt.setString(1, solicitud.getCodigo());
stmt.setString(2, solicitud.getDescripcion());
stmt.setString(3, solicitud.getEstado().name());
stmt.setLong(4, solicitud.getSolicitanteId());
// ... tedioso, frágil, propenso a errores
```

Un ORM permite trabajar directamente con objetos:

```
// ✅ Con ORM: trabajar con objetos
entityManager.persist(solicitudEntity); // El ORM genera el SQL automáticamente
```

Beneficios de un ORM:

Productividad: elimina SQL repetitivo (CRUD)

Portabilidad: cambiar de H2 a PostgreSQL sin reescribir consultas

Seguridad: previene inyecciones SQL

Mantenibilidad: los cambios en el modelo se reflejan automáticamente en las consultas

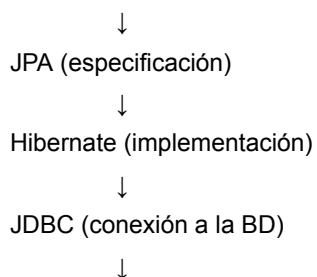
JPA (Jakarta Persistence API)

JPA es la **especificación estándar** de Java para ORM. Define un conjunto de interfaces, anotaciones y reglas que cualquier implementación debe seguir. JPA en sí **no hace nada** — es un contrato.

Hibernate

Hibernate es la **implementación más popular** de JPA. Es el motor que realmente ejecuta las traducciones objeto-relacional. Cuando usamos Spring Data JPA, Hibernate trabaja internamente.

Tu código → usa anotaciones JPA (@Entity, @Column...)



💡💡 **Analogía:** JPA es como una interfaz Java (`SolicitudRepository`) y Hibernate es como la implementación concreta (`SolicitudJpaRepository`). El mismo principio que aplicamos con el patrón `Repository`.

Anotaciones JPA para Mapeo de Entidades

JPA utiliza **anotaciones** para indicar cómo se traduce cada elemento del mundo Java al mundo relacional. A continuación se describen las que usaremos en esta guía:

Anotaciones de Entidad y Tabla

Anotación	Propósito	Ejemplo
<code>@Entity</code>	Marca una clase como entidad persistente	<code>@Entity public class SolicitudEntity</code>
<code>@Table(name = "...")</code>	Nombre de la tabla en la BD	<code>@Table(name = "solicitudes")</code>
<code>@Id</code>	Campo que es la clave primaria	<code>@Id private Long id</code>
<code>@GeneratedValue</code>	La BD genera el valor del ID automáticamente	<code>@GeneratedValue(strategy = GenerationType.IDENTITY)</code>

`@Entity` // → "Esta clase se mapea a una tabla" `@Table(name = "solicitudes")` // → "La tabla se llama 'solicitudes'" `public class SolicitudEntity {`

`@Id` // → "Este campo es la clave primaria"

`@GeneratedValue(strategy = GenerationType.IDENTITY)` // → "La BD genera el valor" `private Long id;`

`}`

💡💡 **Nota:** JPA requiere un **constructor vacío** (sin argumentos) en cada entidad. Esto es porque Hibernate crea instancias usando reflexión y luego inyecta los valores desde la BD.

Anotaciones de Columna

Anotación	Propósito	Ejemplo
<code>@Column</code>	Configura el mapeo de un campo a una columna	<code>@Column(name = "codigo", nullable = false)</code>

nullable	Si la columna acepta valores NULL	nullable = false → NOT NULL
unique	Si el valor debe ser único en la tabla	unique = true → UNIQUE constraint
length	Longitud máxima para String	length = 1000 → VARCHAR(1000)

```
@Column(name = "codigo", unique = true, nullable = false, length = 20)
private String codigo;
// → columna: codigo VARCHAR(20) NOT NULL UNIQUE
```

```
@Column(name = "descripcion", nullable = false, length = 1000)
private String descripcion;
// → columna: descripcion VARCHAR(1000) NOT NULL
```

Mapeo de Enums

Los enums de Java no existen como tipo en bases de datos relacionales. `@Enumerated` define cómo se almacenan:

Estrategia	Almacena	Ejemplo en BD	Ventaja
EnumType.STRING	El nombre del enum	'REGISTRADA'	Legible, resistente a reordenamientos
EnumType.ORDINAL	La posición (0, 1, 2...)	0	Más compacto, pero frágil

```
@Enumerated(EnumType.STRING) // → Almacena "REGISTRADA", "CLASIFICADA", etc.
@Column(name = "estado", nullable = false, length = 20)
private EstadoSolicitudEnum estado;
```

 **Recomendación:** Usar siempre EnumType.STRING . Con ORDINAL , si alguien reordena los valores del enum, los datos existentes en la BD se corrompen silenciosamente.

Objetos Embebidos (Value Objects en JPA)

Cuando un Value Object tiene múltiples campos (como Prioridad con nivel y justificación), no queremos crear una tabla separada — queremos que sus campos se "aplasten" como columnas adicionales en la tabla principal. Para esto usamos `@Embedded` y `@Embeddable` :

```
// En la entidad principal:
@Embedded // → "Incluye los campos de este objeto como columnas"
private PrioridadEmbeddable prioridad;
```

```
// La clase embebida:
@Embeddable
public record PrioridadEmbeddable(
    @Column(name = "prioridad_nivel")
    NivelPrioridadEnum nivel,
```

```
@Column(name = "prioridad_justificacion")
String justificacion
) {}
```

Resultado en la tabla solicitudes :

	id	codigo	prioridad_nivel	prioridad_justificacion	...
	1	SOL-2024-00001	MEDIA	Documentación completa	

❖❖ **Conexión con DDD:** @Embeddable es la forma natural de persistir **Value Objects** del dominio. El VO no tiene identidad propia, así que no necesita su propia tabla — sus campos se embeben en la tabla de la entidad que lo contiene.

Relaciones entre Entidades

Cuando una entidad contiene una **colección** de otras entidades (como Solicitud contiene una lista de EventoHistorial), necesitamos modelar una relación:

Anotación	Significado	Ejemplo
@OneToMany	Una entidad tiene muchas de otra	Una solicitud tiene muchos eventos
@ManyToOne	Muchas entidades pertenecen a una	Muchos eventos pertenecen a una solicitud
@JoinColumn	Columna FK que establece la relación	solicitud_id en tabla eventos_historial

```
// En SolicitudEntity:
@OneToMany(cascade = CascadeType.ALL, orphanRemoval = true, fetch = FetchType.LAZY)
@JoinColumn(name = "solicitud_id")
@OrderBy("secuencia ASC")
private List<EventoHistorialEntity> historial = new ArrayList<>();
```

Desglosemos los atributos:

Atributo	Significado
cascade = CascadeType.ALL	Cuando guardo/elimino una solicitud, también guardo/elimino sus eventos
orphanRemoval = true	Si un evento se quita de la lista, se elimina de la BD
fetch = FetchType.LAZY	Los eventos NO se cargan hasta que alguien acceda a getHistorial()

@JoinColumn(name = "solicitud_id")	La tabla eventos_historial tiene una columna solicitud_id como FK
@OrderBy("secuencia ASC")	Los eventos se ordenan por secuencia al cargarlos

Resultado en la base de datos:

tabla: solicitudes tabla: eventos historial

Diagram illustrating the structure of a sequence (secuencia) and its associated actions (accion) and identifiers (id, codigo, id, solicitud_id).

secuencia	accion	id	codigo	...	id	solicitud_id
1	CREAC.	1	2	1	2	CLASIF.
2	3	2	1	CREAC.	2	2

🔗🔗 **Conexión con DDD:** En la Guía 05 definimos que EventoHistorial **pertenece** al agregado Solicitud y solo se accede a través de la raíz. @OneToMany con CascadeType.ALL refleja exactamente esa regla: los eventos del historial viven y mueren con su solicitud.

Estrategia de Carga: Lazy vs. Eager

Estrategia	Comportamiento	Cuándo usar
FetchType.LAZY	Carga los datos solo cuando se acceden	Por defecto para colecciones (@OneToMany)

Estrategia	Comportamiento	Cuándo usar
FetchType.EAGER	Carga los datos inmediatamente con la entidad padre	Solo cuando SIEMPRE necesitas los datos

```
// LAZY: los eventos NO se cargan al hacer findById(1)
```

```
// Hasta que alguien llame getHistorial()
```

```
@OneToMany(fetch = FetchType.LAZY)
```

```
private List<EventoHistorialEntity> historial;
```

⚠️ Precaución: Usar EAGER en colecciones puede causar severos problemas de rendimiento (el problema N+1). Siempre preferir LAZY y cargar explícitamente cuando se necesite.

Repository Pattern

Patrón que abstrae el acceso a datos, mediando entre el dominio y la capa de persistencia. Ya lo estudiamos

en la Guía 06 como interfaz; en esta guía lo implementamos con una tecnología real.

Referencia: Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.

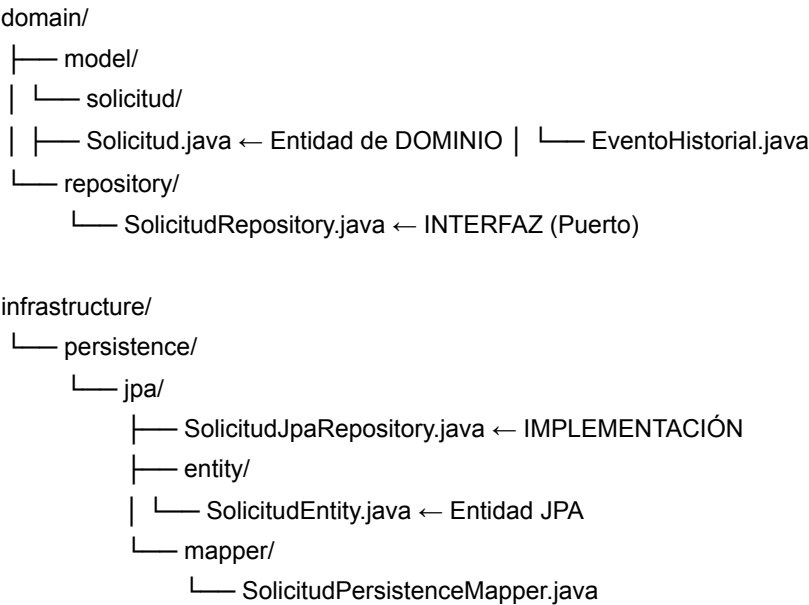
CONTEXTUALIZACIÓN TEÓRICA

¿Por qué H2 para este curso?

Aspecto	H2 (embebida)	PostgreSQL / MySQL (servidor)
Instalación	✓ Ninguna (incluida como dependencia)	✗ Requiere instalar y configurar servidor
Configuración	✓ Automática con Spring Boot	✗ Configuración de conexión, usuarios, permisos
Consola visual	✓ Web integrada en /h2-console	Requiere herramientas externas (pgAdmin, Workbench)
Modelo	Relacional (tablas, SQL estándar)	Relacional (tablas, SQL estándar)
Ideal para	Desarrollo y testing	Producción

💡💡 **Nota:** H2 habla SQL estándar. Lo que aprendas aquí con H2 se aplica directamente a PostgreSQL o MySQL en producción. El cambio de base de datos solo requiere modificar la configuración en `application.yml` — sin cambiar una línea de código Java.

Arquitectura de Persistencia



PARTE 1: CONFIGURACIÓN H2

Dependencias Gradle

```
dependencies {  
    // Spring Data JPA  
    implementation 'org.springframework.boot:spring-boot-starter-data-jpa'  
  
    // H2 Database  
    runtimeOnly 'com.h2database:h2'  
  
    // Validación  
    implementation 'org.springframework.boot:spring-boot-starter-validation' }
```

Configuración application.yml

```
spring:  
  application:  
    name: triage-system  
  
# Configuración H2  
datasource:  
  url: jdbc:h2:mem:triagedb # Base de datos en memoria # url: jdbc:h2:file:./data/triagedb # O  
  persiste en archivo driver-class-name: org.h2.Driver  
  username: sa  
  password:  
  
# Configuración JPA/Hibernate  
jpa:  
  database-platform: org.hibernate.dialect.H2Dialect  
  hibernate:  
    ddl-auto: create-drop # create-drop para desarrollo, validate para producción show-sql: true # Mostrar SQL  
    en consola (desarrollo)  
  properties:  
    hibernate:  
      format_sql: true # Formatear SQL  
      use_sql_comments: true  
  
# Consola H2 (interfaz web)  
h2:  
  console:  
    enabled: true  
    path: /h2-console # Acceder en http://localhost:8080/h2-console settings:  
    web-allow-others: false
```

Acceder a la Consola H2

1. Ejecutar la aplicación Spring Boot
2. Abrir navegador: `http://localhost:8080/h2-console`
3. Configurar conexión:

JDBC URL: `jdbc:h2:mem:triagedb`

User Name: `sa`

Password: (vacío)

4. Click en "Connect"

PARTE 2: INTERFAZ REPOSITORY (DOMINIO)

Definir Contrato en el Dominio

En la Guía 06 ya definimos una interfaz básica en el dominio, y en la Guía 09 la adaptamos a convenciones Spring Data (`save` , `findById` , `findByEstado`). Ahora la extendemos con consultas adicionales que el dominio necesita:

```
package co.edu.uniquindio.solicitudes.domain.repository;
```

```
import co.edu.uniquindio.solicitudes.domain.entity.*; import
co.edu.uniquindio.solicitudes.domain.valueobject.*; import java.util.List;
import java.util.Optional;
```

```
/**
```

```
 * Puerto (interface) definido en el DOMINIO
```

```
 * No conoce nada de JPA, Hibernate, H2, etc.
```

```
 *
```

```
 * Describe QUÉ operaciones necesita el dominio, * no CÓMO
se implementan.
```

```
*/
```

```
public interface SolicitudRepository {
```

```
    // Operaciones CRUD básicas (ya existían en Guías 06/09) Solicitud
    save(Solicitud solicitud);
```

```
    Optional<Solicitud> findById(Long id);
```

```
    List<Solicitud> findAll();
```

```
    void deleteById(Long id);
```

```
    boolean existsById(Long id);
```

```
    // Consultas ya existentes (Guía 06/09)
```

```
    List<Solicitud> findByEstado(EstadoSolicitud estado);
```

```
Optional<Solicitud> findByCodigo(String codigo);
```

```
// Nuevas consultas para esta guía:
```

```
// A medida que el sistema crece, el dominio necesita // consultas más específicas.
```

```
List<Solicitud> findBySolicitanteId(Long solicitanteId);
```

```
List<Solicitud> findByEstadoAndPrioridad(
```

```
    EstadoSolicitud estado,
```

```
    NivelPrioridad prioridad
```

```
);
```

```
long countByEstado(EstadoSolicitud estado);
```

```
List<Solicitud> findSolicitudesPendientesDeAsignacion(); }
```

💡💡 **Sobre el tipo de ID (Long):** En las guías 04-05 usamos SolicitudId como Value Object y en la Guía 09 se usó String . Al integrar con una base de datos relacional, el ID se convierte en un Long autogenerated por la base de datos (@GeneratedValue). El CodigoSolicitud (ej: SOL-2024-00123) sigue existiendo como **identificador de negocio**: es el código que el usuario ve. El Long id es el **identificador técnico** que usa la base de datos internamente.

Nota: Esta interfaz está en domain/repository/ , NO importa nada de JPA.

PARTE 3: ENTIDAD JPA

¿Por qué crear SolicitudEntity en lugar de anotar Solicitud directamente?

Esta es una pregunta natural: si JPA mapea clases Java a tablas, ¿por qué no anotar directamente la entidad de dominio Solicitud con @Entity , @Column , etc.? ¿No estamos duplicando código?

Veamos qué pasaría si intentáramos usar Solicitud del dominio como entidad JPA:

```
// ❌ Problema: Intentar usar la entidad de dominio directamente con JPA
```

```
@Entity // Ahora el dominio DEPENDE de jakarta.persistence
```

```
@Table(name = "solicitudes")
```

```
public class Solicitud {
```

```
    @Id
```

```
    @GeneratedValue // Ahora el dominio SABE que hay una base de datos
```

```
    private Long id;
```

```
// JPA requiere constructor vacío — pero el dominio lo prohíbe
```

```
// porque una Solicitud sin datos es inválida
```

```
public Solicitud() {} // ❌ Rompe invariantes del dominio
```

```

// JPA necesita setters — pero el dominio los prohíbe
// porque el estado solo debe cambiar a través de métodos de negocio
public void setEstado(EstadoSolicitud estado) { // ❌ Bypass de reglas
    this.estado = estado;
}

// El dominio tiene comportamiento rico (cerrar, clasificar, asignar)
// JPA no necesita nada de esto — solo datos
public void cerrar() { ... }
}

```

Los problemas son fundamentales:

Aspecto	La entidad de Dominio	La entidad JPA necesita
Constructor	Valida invariantes al crear	Constructor vacío (para reflexión)
Setters	Prohibidos (estado se cambia vía métodos de negocio)	Obligatorios (para que Hibernate inyecte datos)
Anotaciones	Cero dependencias de frameworks	@Entity , @Column , @OneToMany ...
Comportamiento	Métodos ricos: cerrar() , clasificar()	Solo datos (getters/setters)
Propósito	Proteger reglas de negocio	Describir la estructura de la tabla

❖❖ **Principio clave:** Si anotamos Solicitud con JPA, el dominio queda **acoplado a la tecnología de persistencia**. Si mañana cambiamos de JPA a otro mecanismo (MongoDB, archivos JSON, un servicio externo), tendríamos que modificar la clase más importante del dominio.

Por esta razón creamos un **modelo de persistencia separado**:

```

domain/entity/Solicitud.java ← Comportamiento, reglas, sin JPA
infrastructure/persistence/entity/SolicitudEntity.java ← Datos, estructura de tabla, con JPA
infrastructure/persistence/mapper/SolicitudPersistenceMapper.java ← Traduce entre ambos

```

❖❖ **Nota pragmática:** En muchos proyectos de la industria, especialmente con equipos pequeños o CRUD simples, **sí se anota la entidad de dominio directamente con JPA**. Es un trade-off válido entre pureza arquitectónica y productividad. En este curso separamos los modelos para que entiendas el principio — en la práctica profesional, decides según el contexto.

Modelo de Persistencia (Infraestructure)

```

package co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa.entity;

import jakarta.persistence.*;
import lombok.Getter;

```

```

import lombok.Setter;
import lombok.NoArgsConstructor;
import lombok.AllArgsConstructor;
import java.time.LocalDateTime;
import java.util.ArrayList;
import java.util.List;

/**
 * Entidad JPA (modelo de persistencia)
 * Vive en INFRASTRUCTURE, no en DOMAIN
 *
 * Lombok genera automáticamente:
 * - @Getter → todos los getters
 * - @Setter → todos los setters
 * - @NoArgsConstructor → constructor vacío (requerido por JPA) */
@Entity
@Table(name = "solicitudes")
@Getter @Setter @NoArgsConstructor
public class SolicitudEntity {

    @Id
    @GeneratedValue(strategy = GenerationType.IDENTITY)
    private Long id;

    @Column(name = "codigo", unique = true, nullable = false, length = 20) private String codigo;

    @Column(name = "tipo_solicitud_id", nullable = false)
    private Long tipoSolicitudId;

    @Column(name = "descripcion", nullable = false, length = 1000) private String
    descripcion;

    @Enumerated(EnumType.STRING)
    @Column(name = "estado", nullable = false, length = 20)
    private EstadoSolicitudEnum estado;

    // Objeto embebido para Prioridad (Value Object)
    @Embedded
    private PrioridadEmbeddable prioridad;
    @Column(name = "solicitante_id", nullable = false)
    private Long solicitanteId;

    @Column(name = "responsable_id")
    private Long responsableId;

    @Enumerated(EnumType.STRING)
    @Column(name = "canal_origen", length = 30)
    private CanalOrigenEnum canalOrigen;

    // Relación One-to-Many con EventoHistorial

```

```

@ElementCollection(fetch = FetchType.LAZY)
@CollectionTable(name = "eventos_historial", joinColumns = @JoinColumn(name = "solicitud_id" private
List<EventoHistorialEmbeddable> historial = new ArrayList<>();

@Column(name = "fecha_creacion", nullable = false)
private LocalDateTime fechaCreacion;

@Column(name = "fecha_actualizacion")
private LocalDateTime fechaActualizacion;
}

/**
 * Objeto embebido para Prioridad (Value Object)
 */
@Embeddable
@Getter @Setter @NoArgsConstructor @AllArgsConstructor
class PrioridadEmbeddable {

    @Enumerated(EnumType.STRING)
    @Column(name = "prioridad_nivel", length = 20)
    private NivelPrioridadEnum nivel;

    @Column(name = "prioridad_justificacion", length = 500)
}
> **💡 Nota Arquitectónica: Colecciones de Value Objects (`@ElementCollection`)** >
> En la capa de dominio, `EventoHistorial` es un **Value Object puro** (no tiene identidad propia >
> **¿Por qué esto y no un `@OneToMany` estricto con una Entidad (`@Entity`)?** > Desde un punto de vista puramente pedagógico
y conceptual, crear un `@Entity` artificial con un >
> *Nota: En escenarios de alto rendimiento o colecciones masivas, `@ElementCollection` puede ser

/**
 * Embeddable JPA para la colección de EventoHistorial
 */
@Embeddable
public record EventoHistorialEmbeddable(

    @Column(name = "fecha_hora", nullable = false)
    LocalDateTime fechaHora,

    @Enumerated(EnumType.STRING)
    @Column(name = "accion", nullable = false, length = 30)
    TipoAccionEnum accion,

    @Column(name = "responsable_id", nullable = false)
    Long responsableId,

    @Column(name = "observacion", length = 500)
    String observacion,

    @Enumerated(EnumType.STRING)
    @Column(name = "estado_anterior", length = 20)

```

```

EstadoSolicitudEnum estadoAnterior,

@Enumerated(EnumType.STRING)
@Column(name = "estado_nuevo", length = 20)
EstadoSolicitudEnum estadoNuevo
) {}

/**
 * Enums para JPA (réplicas de los del dominio)
 *
 * ¿Por qué replicar los enums en lugar de usar los del dominio directamente?
 * Opción A (la que usamos aquí): Enums separados en infrastructure. * - Ventaja: El dominio NO
depende de la capa de persistencia en absoluto. * - Desventaja: Requiere mapeo bidireccional con
valueOf().
 * Opción B (alternativa válida): Usar los enums del dominio directamente * en las entidades JPA
con @Enumerated(EnumType.STRING). * - Ventaja: Menos código, sin mapeo.
 * - Desventaja: Las entidades JPA importan clases del dominio.
 * Ambas opciones son válidas en la industria. En este curso separamos * para ser coherentes
con el principio de independencia de capas. */
enum EstadoSolicitudEnum {
    REGISTRADA, CLASIFICADA, ASIGNADA, EN_ATENCION, ATENDIDA, CERRADA }

enum NivelPrioridadEnum {
    CRITICA, ALTA, MEDIA, BAJA
}
enum CanalOrigenEnum {
    CSU, CORREO_ELECTRONICO, SAC, PRESENCIAL, TELEFONICO, WEB
}

enum TipoAccionEnum {
    CREACION, CLASIFICACION, ASIGNACION, CAMBIO_ESTADO, PRIORIZACION, CIERRE
}

```

PARTE 4: MAPPER PERSISTENCIA CON MAPSTRUCT

💡💡 **Conexión con la Guía 08:** En la Guía 08 usamos MapStruct para mapear entre DTOs REST y entidades de dominio. Ahora usamos MapStruct para un propósito similar pero en la dirección opuesta: mapear entre entidades de dominio y entidades JPA (persistencia).

Enfoque Híbrido: MapStruct + Método Manual

El mapeo de persistencia tiene dos direcciones con complejidades muy diferentes:

Dirección	Propósito	Complejidad	Enfoque
-----------	-----------	-------------	---------

Domain → Entity	Guardar en BD	Media: campos, enums, embeddable	✓ MapStruct
Entity → Domain	Cargar desde BD	Alta: factory method, repositorios	✓ Método default manual

¿Por qué no usar MapStruct para toDomain() ? Porque requiere:

- Buscar Usuario y TipoSolicitud en sus repositorios
- Reconstruir Value Objects (CodigoSolicitud , Prioridad)
- Invocar un método factory (Solicitud.reconstruirDesdeDB())

MapStruct podría manejarlo con @Context y @ObjectFactory , pero las anotaciones resultantes serían más confusas que el código manual. Usamos un enfoque **pragmático**: lo que MapStruct simplifica, lo delegamos; lo que requiere lógica compleja, lo escribimos.

Interfaz MapStruct

```
package co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa.mapper;

import co.edu.uniquindio.solicitudes.domain.entity.*;
import co.edu.uniquindio.solicitudes.domain.valueobject.*;
import co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa.entity.*; import org.mapstruct.*;

import java.util.List;
import java.util.stream.Collectors;

/**
 * Mapper entre Entidad de Dominio y Entidad JPA
 * Usa MapStruct para las conversiones automáticas (Domain → Entity)
 * y un método default para la reconstrucción compleja (Entity → Domain)
 */
@Mapper(componentModel = "spring")
public interface SolicitudPersistenceMapper {

    // =====
    // DOMAIN → ENTITY (MapStruct genera la implementación)
    // =====

    @Mapping(target = "codigo", source = "codigo.valor")
    @Mapping(target = "tipoSolicitudId", source = "tipo.id")
    @Mapping(target = "estado", source = "estado")
    @Mapping(target = "prioridad", source = "prioridad")
    @Mapping(target = "solicitanteId", source = "solicitante.id")
    @Mapping(target = "responsableId", source = "responsable.id")
    @Mapping(target = "canalOrigen", source = "canalOrigen")
    @Mapping(target = "historial", source = "historialCompleto")
    SolicitudEntity toEntity(Solicitud solicitud);
}
```



```

// Mapeo del Value Object Prioridad → Embeddable
@Mapping(target = "nivel", source = "nivel")
@Mapping(target = "justificacion", source = "justificacion")
PrioridadEmbeddable toPrioridadEmbeddable(Prioridad prioridad);

// Mapeo de EventoHistorial → EventoHistorialEntity
@Mapping(target = "responsableId", source = "responsable.id")
EventoHistorialEmbeddable toEventoHistorialEmbeddable(EventoHistorial evento); List<EventoHistorialEmbeddable>

toEventoHistorialEmbeddableList(List<EventoHistorial> eventos)

// Conversión automática de Enums (mismo nombre de valores)
// MapStruct resuelve esto automáticamente porque los valores
// de los enums del dominio y de JPA tienen los mismos nombres. // Ejemplo:
EstadoSolicitud.REGISTRADA → EstadoSolicitudEnum.REGISTRADA EstadoSolicitudEnum
toEstadoEnum(EstadoSolicitud estado); NivelPrioridadEnum
toNivelPrioridadEnum(NivelPrioridad nivel); CanalOrigenEnum toCanalOrigenEnum(CanalOrigen
canal);
TipoAccionEnum toTipoAccionEnum(TipoAccion accion);

// Conversión inversa de Enums (Entity → Domain)
EstadoSolicitud toEstadoDomain(EstadoSolicitudEnum estado); NivelPrioridad
toNivelPrioridadDomain(NivelPrioridadEnum nivel); CanalOrigen
toCanalOrigenDomain(CanalOrigenEnum canal); TipoAccion
toTipoAccionDomain(TipoAccionEnum accion);

// ===== //
ENTITY → DOMAIN (método default — lógica compleja manual) //
=====

/**
 * Reconstruir la entidad de dominio desde la entidad JPA. *
 * Este método NO puede generarse automáticamente porque: * 1. Necesita objetos externos
(TipoSolicitud, Usuario) que no están * en la entidad JPA (solo sus IDs)
 * 2. Debe invocar el factory method Solicitud.reconstruirDesdeDB() * 3. Debe reconstruir
Value Objects (CodigoSolicitud, Prioridad) */
default Solicitud toDomain(
    SolicitudEntity entity,
    TipoSolicitud tipo,
    Usuario solicitante,
    Usuario responsable) {

    if (entity == null) {
        return null;
    }

    // 1. Reconstruir Value Objects
    CodigoSolicitud codigo = new CodigoSolicitud(entity.getCodigo()); EstadoSolicitud

```

```
estado = toEstadoDomain(entity.getEstado()); CanalOrigen canal =  
toCanalOrigenDomain(entity.getCanalOrigen());
```

```
Prioridad prioridad = null;  
if (entity.getPrioridad() != null) {  
    prioridad = new Prioridad(  
        toNivelPrioridadDomain(entity.getPrioridad().nivel()),  
        entity.getPrioridad().justificacion()  
    );  
}
```

```
// 2. Reconstruir historial
```

```
List<EventoHistorial> historial = entity.getHistorial().stream()  
    .map(this::toEventoHistorialDomain)  
    .collect(Collectors.toList());
```

```
// 3. Usar método factory de reconstrucción
```

```
//
```

```
// ¿Por qué un método factory y no un constructor?
```

```
// La entidad Solicitud del dominio (Guía 05) no expone setters // para proteger sus  
invariantes. Necesitamos un método especial // que permita reconstruir una solicitud  
desde datos persistidos, // sin pasar por las validaciones de creación.
```

```
//
```

```
// En la clase Solicitud del dominio, debe existir algo como: //
```

```
// public static Solicitud reconstruirDesdeDB(  
// Long id, CodigoSolicitud codigo, TipoSolicitud tipo, // String descripcion, Usuario solicitante,  
// Usuario responsable, // EstadoSolicitud estado, Prioridad prioridad, CanalOrigen canal, //  
// List<EventoHistorial> historial,  
// LocalDateTime fechaCreacion, LocalDateTime fechaActualizacion) { // // Asigna todos los campos  
// directamente, sin validaciones // }
```

```
//
```

```
List<EventoHistorial> historial,
```

```
// LocalDateTime fechaCreacion, LocalDateTime fechaActualizacion) { // // Asigna todos los campos  
directamente, sin validaciones // }
```

```
//
```

```
return Solicitud.reconstruirDesdeDB(  
    entity.getId(),  
    codigo,  
    tipo,  
    entity.getDescripcion(),  
    solicitante,  
    responsable,  
    estado,  
    prioridad,  
    canal,  
    historial,  
    entity.getFechaCreacion(),  
    entity.getFechaActualizacion()  
);
```

```
}
```

```
/**
```

```
* EJERCICIO: Completar este método default.
```

```
* Debe reconstruir un EventoHistorial de dominio a partir de la entidad JPA. *
```

```
* Pistas:
```

```

* - Usar los métodos de conversión de enums ya definidos arriba * (toEstadoDomain,
toTipoAccionDomain)
* - El responsableId es solo un Long — puede necesitar un repositorio
* para reconstruir un Usuario completo, o bien almacenar solo el ID
*/
default EventoHistorial toEventoHistorialDomain(EventoHistorialEmbeddable embeddable) { // TODO: Implementar
    como ejercicio
    throw new UnsupportedOperationException("Implementar como ejercicio");
}
}

```

¿Qué genera MapStruct automáticamente?

En tiempo de compilación, MapStruct genera una clase SolicitudPersistenceMapperImpl que incluye:

```

// Generado automáticamente por MapStruct (NO escribir manualmente)
@Component
public class SolicitudPersistenceMapperImpl implements SolicitudPersistenceMapper {

    @Override
    public SolicitudEntity toEntity(Solicitud solicitud) {
        if (solicitud == null) {
            return null;
        }
        SolicitudEntity entity = new SolicitudEntity();
        entity.setId(solicitud.getId());
        entity.setCodigo(solicitud.getCodigo().getValor()); // MapStruct navega el path
        entity.setTipoSolicitudId(solicitud.getTipo().getId());
        entity.setEstado(toEstadoEnum(solicitud.getEstado())); // Usa el método de conversión
        entity.setPrioridad(toPrioridadEmbeddable(solicitud.getPrioridad()));
        entity.setSolicitanteId(solicitud.getSolicitante().getId());
        // ... etc (todos los campos mapeados automáticamente)
        return entity;
    }

    @Override
    public EstadoSolicitudEnum toEstadoEnum(EstadoSolicitud estado) {
        if (estado == null) return null;
        return EstadoSolicitudEnum.valueOf(estado.name()); // Nombres iguales → automático }

    // Los métodos default (toDomain, toEventoHistorialDomain)
    // se heredan directamente de la interface — no se sobrescriben
}

```

💡💡 **Resultado:** ~60% menos código escrito manualmente. El mapeo toEntity que antes eran 40 líneas ahora son 7 anotaciones @Mapping . Las conversiones de enums (antes con valueOf() manual) son automáticas. Solo el toDomain() se escribe a mano porque su complejidad lo justifica.

PARTE 5: IMPLEMENTACIÓN REPOSITORY

Spring Data JPA Repository

```
package co.edu.uniquindio.solicitudes.infraestructure.persistence.jpa;

import co.edu.uniquindio.solicitudes.infraestructure.persistence.jpa.entity.SolicitudEntity; import
co.edu.uniquindio.solicitudes.infraestructure.persistence.jpa.entity.EstadoSolicitudEnum; import
co.edu.uniquindio.solicitudes.infraestructure.persistence.jpa.entity.NivelPrioridadEnum; import
org.springframework.data.jpa.repository.JpaRepository;
import org.springframework.data.jpa.repository.Query;
import org.springframework.data.repository.query.Param;
import java.util.List;
import java.util.Optional;

/**
 * Interfaz Spring Data JPA - genera implementación automática
 * Solo maneja SolicitudEntity (modelo de persistencia)
 */
interface SolicitudJpaDataRepository extends JpaRepository<SolicitudEntity, Long> {

    // Query methods (nombrados por convención)
    List<SolicitudEntity> findByEstado(EstadoSolicitudEnum estado);

    List<SolicitudEntity> findBySolicitanteld(Long solicitanteld);

    Optional<SolicitudEntity> findByCodigo(String codigo);

    // Query methods compuestos
    List<SolicitudEntity> findByEstadoAndPrioridadNivel(
        EstadoSolicitudEnum estado,
        NivelPrioridadEnum prioridad
    );

    long countByEstado(EstadoSolicitudEnum estado);

    // Consulta personalizada con JPQL
    @Query("SELECT s FROM SolicitudEntity s WHERE s.estado IN ('REGISTRADA', 'CLASIFICADA')")
    List<SolicitudEntity> findSolicitudesPendientesDeAsignacion();

    // Consulta con parámetros
    @Query("SELECT s FROM SolicitudEntity s WHERE s.solicitanteld = :userId AND s.estado = :estado List<SolicitudEntity>")
    findByUserAndStatus(
        @Param("userId") Long userId,
        @Param("estado") EstadoSolicitudEnum estado
    );
}
```

Adaptador que Implementa el Puerto del Dominio

```

package co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa;

import co.edu.uniquindio.solicitudes.domain.entity.*;
import co.edu.uniquindio.solicitudes.domain.valueobject.*;
import co.edu.uniquindio.solicitudes.domain.entity.Usuario;
import co.edu.uniquindio.solicitudes.domain.repository.SolicitudRepository; import
co.edu.uniquindio.solicitudes.domain.repository.UsuarioRepository;
import co.edu.uniquindio.solicitudes.domain.repository.TipoSolicitudRepository; import
co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa.entity.*; import
co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa.mapper.SolicitudPersistenceMa import
org.springframework.stereotype.Repository;
import org.springframework.transaction.annotation.Transactional;
import java.util.List;
import java.util.Optional;
import java.util.stream.Collectors;

/**
 * ADAPTADOR: Implementa el puerto (interfaz) del dominio
 * usando H2/JPA como tecnología de persistencia.
 *
 * IMPORTANTE: Este adaptador depende de UsuarioRepository y TipoSolicitudRepository * para poder
 * reconstruir las entidades de dominio completas (con sus relaciones). * Esto significa que ANTES de implementar
 * este repositorio, deben existir
 * implementaciones JPA de UsuarioRepository y TipoSolicitudRepository.
 * Créelos siguiendo el mismo patrón mostrado aquí.
 */
@Repository
@Transactional
public class SolicitudJpaRepository implements SolicitudRepository {

    private final SolicitudJpaDataRepository dataRepository;
    private final SolicitudPersistenceMapper mapper;
    private final UsuarioRepository usuarioRepository;
    private final TipoSolicitudRepository tipoSolicitudRepository;

    public SolicitudJpaRepository(
        SolicitudJpaDataRepository dataRepository,
        SolicitudPersistenceMapper mapper,
        UsuarioRepository usuarioRepository,
        TipoSolicitudRepository tipoSolicitudRepository) {
        this.dataRepository = dataRepository;
        this.mapper = mapper;
        this.usuarioRepository = usuarioRepository;
        this.tipoSolicitudRepository = tipoSolicitudRepository;
    }

    @Override
    public Solicitud save(Solicitud solicitud) {
        // 1. Convertir Dominio → Entity

```

```

        SolicitudEntity entity = mapper.toEntity(solicitud);

        // 2. Guardar en H2
        SolicitudEntity savedEntity = dataRepository.save(entity);

        // 3. Convertir Entity → Dominio
        return toDomainWithRelations(savedEntity);
    }

    @Override
    @Transactional(readOnly = true)
    public Optional<Solicitud> findById(Long id) {
        return dataRepository.findById(id)
            .map(this::toDomainWithRelations);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Solicitud> findAll() {
        return dataRepository.findAll().stream()
            .map(this::toDomainWithRelations)
            .collect(Collectors.toList());
    }

    @Override
    public void deleteById(Long id) {
        dataRepository.deleteById(id);
    }

    @Override
    @Transactional(readOnly = true)
    public boolean existsById(Long id) {
        return dataRepository.existsById(id);
    }

    @Override
    @Transactional(readOnly = true)
    public List<Solicitud> findByEstado(EstadoSolicitud estado) { EstadoSolicitudEnum estadoEnum =
        EstadoSolicitudEnum.valueOf(estado.name()); return
        dataRepository.findByEstado(estadoEnum).stream()
            .map(this::toDomainWithRelations)
            .collect(Collectors.toList());
    }

    @Override
    @Transactional(readOnly = true)
    public List<Solicitud> findById(Long solicitudId) { return
        dataRepository.findById(solicitudId).stream()
            .map(this::toDomainWithRelations)
            .collect(Collectors.toList());
    }

```

```

@Override
@Transactional(readOnly = true)
public List<Solicitud> findByEstadoAndPrioridad(
    EstadoSolicitud estado,
    NivelPrioridad prioridad) {

    EstadoSolicitudEnum estadoEnum = EstadoSolicitudEnum.valueOf(estado.name()); NivelPrioridadEnum
    prioridadEnum = NivelPrioridadEnum.valueOf(prioridad.name());

    return dataRepository
        .findByEstadoAndPrioridadNivel(estadoEnum, prioridadEnum)
        .stream()
        .map(this::toDomainWithRelations)
        .collect(Collectors.toList());
}

@Override
@Transactional(readOnly = true)
public Optional<Solicitud> findByCodigo(String codigo) {
    return dataRepository.findByCodigo(codigo)
        .map(this::toDomainWithRelations);
}

@Override
@Transactional(readOnly = true)
public long countByEstado(EstadoSolicitud estado) {
    EstadoSolicitudEnum estadoEnum = EstadoSolicitudEnum.valueOf(estado.name()); return
    dataRepository.countByEstado(estadoEnum);
}

@Override
@Transactional(readOnly = true)
public List<Solicitud> findSolicitudesPendientesDeAsignacion() { return
    dataRepository.findSolicitudesPendientesDeAsignacion().stream()
    .map(this::toDomainWithRelations)
    .collect(Collectors.toList());
}

/**
 * Método auxiliar: Reconstruir entidad de dominio con relaciones
 * Carga Usuario, TipoSolicitud desde sus repositorios
 */
private Solicitud toDomainWithRelations(SolicitudEntity entity) { TipoSolicitud tipo =
    tipoSolicitudRepository
        .findById(entity.getTipoSolicitudId())
        .orElseThrow(() -> new IllegalStateException(
            "TipoSolicitud no encontrado: " + entity.getTipoSolicitudId()));

    Usuario solicitante = usuarioRepository

```

```

        .findById(entity.getSolicitanteId())
        .orElseThrow(() -> new IllegalStateException(
            "Usuario solicitante no encontrado: " + entity.getSolicitanteId()));

    Usuario responsable = null;
    if (entity.getResponsableId() != null) {
        responsable = usuarioRepository
            .findById(entity.getResponsableId())
            .orElse(null);
    }

    return mapper.toDomain(entity, tipo, solicitante, responsable);
}
}

```

PARTE 6: INICIALIZACIÓN DE DATOS

Script SQL (data.sql)

Crear archivo src/main/resources/data.sql :

```

-- Tipos de Solicitud
INSERT INTO tipos_solicitud (id, codigo, nombre, descripcion) VALUES
(1, 'HOMOL', 'Homologación', 'Solicitud de homologación de asignaturas'), (2, 'CUPO', 'Cupo
Adicional', 'Solicitud de cupo adicional en asignatura'), (3, 'CANCEL', 'Cancelación', 'Solicitud de
cancelación de asignatura');

-- Usuarios iniciales
INSERT INTO usuarios (id, nombre_completo, email, rol) VALUES
(1, 'Juan Pérez', 'juan@uniquindio.edu.co', 'ESTUDIANTE'),
(2, 'María García', 'maria.garcia@uniquindio.edu.co', 'COORDINADOR');

-- Solicitudes de ejemplo
INSERT INTO solicitudes (codigo, tipo_solicitud_id, descripcion, estado, solicitante_id, canal_o ('SOL-2024-00001', 1, 'Solicito
homologación de Programación I', 'REGISTRADA', 1, 'CORREO_ELECTRO ('SOL-2024-00002', 2, 'Requiero cupo adicional en
Bases de Datos', 'CLASIFICADA', 1, 'CSU', NOW(

```

PARTE 7: TESTING

Test del Repository

```

package co.edu.uniquindio.solicitudes.infrastructure.persistence.jpa;

import co.edu.uniquindio.solicitudes.domain.entity.*;
import co.edu.uniquindio.solicitudes.domain.valueobject.*;
import org.junit.jupiter.api.Test;

```



```

import org.springframework.beans.factory.annotation.Autowired;
import org.springframework.boot.test.autoconfigure.orm.jpa.DataJpaTest;
import org.springframework.context.annotation.Import;

import static org.junit.jupiter.api.Assertions.*;

@DataJpaTest // Spring crea H2 en memoria automáticamente
@Import({SolicitudJpaRepository.class, SolicitudPersistenceMapperImpl.class}) // MapStruct genera class
SolicitudJpaRepositoryTest {

    @Autowired
    private SolicitudJpaRepository repository;

    @Test
    void deberiaGuardarYRecuperarSolicitud() {
        // Arrange
        Solicitud solicitud = crearSolicitudMock();

        // Act
        Solicitud guardada = repository.save(solicitud);
        Optional<Solicitud> recuperada = repository.findById(guardada.getId());

        // Assert
        assertTrue(recuperada.isPresent());
        assertEquals(guardada.getCodigo(), recuperada.get().getCodigo());
    }

    @Test
    void deberiaEncontrarPorEstado() {
        // Arrange
        Solicitud s1 = crearSolicitudConEstado(EstadoSolicitud.REGISTRADA);
        Solicitud s2 = crearSolicitudConEstado(EstadoSolicitud.CLASIFICADA);
        repository.save(s1);
        repository.save(s2);

        // Act
        List<Solicitud> registradas = repository.findByEstado(EstadoSolicitud.REGISTRADA);

        // Assert
        assertEquals(1, registradas.size());
        assertEquals(EstadoSolicitud.REGISTRADA, registradas.get(0).getEstado());
    }
}

```

EJERCICIOS PRÁCTICOS

Ejercicio 1: Implementar repositorios adicionales

Siguiendo el mismo patrón de `SolicitudJpaRepository` , implemente:

1. **UsuarioJpaRepository** : que implemente la interfaz `UsuarioRepository` del dominio. 2.

TipoSolicitudJpaRepository : que implemente la interfaz `TipoSolicitudRepository` del dominio.

Para cada uno necesitará:

Una entidad JPA (`UsuarioEntity` , `TipoSolicitudEntity`)

Un mapper de persistencia

Una interfaz Spring Data (`extends JpaRepository<...>`)

Un adaptador que implemente la interfaz del dominio

Ejercicio 2: Verificar datos en consola H2

1. Ejecute la aplicación con los datos iniciales del script `data.sql`

2. Abra la consola H2 en `http://localhost:8080/h2-console`

3. Ejecute las siguientes consultas SQL y verifique los resultados:

```
SELECT * FROM solicitudes;
```

```
SELECT * FROM tipos_solicitud;
```

```
SELECT * FROM usuarios;
```

4. Verifique que las tablas generadas por Hibernate coinciden con las entidades JPA definidas

Ejercicio 3: Agregar Índices

Mejorar performance agregando índices:

```
@Entity
```

```
@Table(name = "solicitudes", indexes = {
```

```
    @Index(name = "idx_estado", columnList = "estado"),
```

```
    @Index(name = "idx_codigo", columnList = "codigo", unique = true), @Index(name =
```

```
    "idx_solicitante", columnList = "solicitante_id")
```

```
})
```

```
public class SolicitudEntity {
```

```
    // ...
```

```
}
```

Ejercicio 4: Implementar Paginación

```
@Override
```

```
public Page<Solicitud> findByEstado(EstadoSolicitud estado, Pageable pageable) { EstadoSolicitudEnum
```

```
    estadoEnum = EstadoSolicitudEnum.valueOf(estado.name()); Page<SolicitudEntity> page =
```

```
    dataRepository.findByEstado(estadoEnum, pageable); return page.map(this::toDomainWithRelations);
```

```
}
```

Ejercicio 5: Flujo end-to-end

Ejecute la aplicación completa y valide el flujo desde HTTP hasta H2:

1. Inicie la aplicación
2. Usando Swagger UI (/swagger-ui.html) o Postman:
 - Cree una nueva solicitud (POST /api/solicitudes)
 - Consulte la solicitud creada (GET /api/solicitudes/{id})
3. Verifique en la consola H2 que la solicitud aparece en la tabla solicitudes 4. Confirme que los Use Cases y Controllers **no fueron modificados** respecto a la Guía 09

PRECAUCIONES Y RECOMENDACIONES

1. **Dominio independiente:** NUNCA importar clases de JPA en el dominio
2. **Transacciones:** Usar @Transactional en los métodos del adaptador 3.
- Lazy Loading:** Tener cuidado con relaciones lazy, cargar lo necesario
4. **ID generado:** El ID es null antes de persistir, se asigna al guardar
5. **H2 en producción:** NO usar H2 en producción, solo desarrollo/testing

EVALUACIÓN O RESULTADO

Al finalizar, los estudiantes deben:

1. ☒ Configurar H2 en Spring Boot
2. ☒ Definir interfaces Repository en el dominio
3. ☒ Implementar repositorios con Spring Data JPA
4. ☒ Crear mappers entre dominio y entidades JPA
5. ☒ Usar la consola H2 para verificar datos
6. ☒ Escribir tests con H2 embebida

PRÓXIMA ACTIVIDAD

Para la **Guía 11 (Consultas)**, preparar:

1. **Explorar:** Consola H2 y las tablas generadas
2. **Leer:** Spring Data JPA reference documentation — sección de query methods
3. **Investigar:** JPQL (Jakarta Persistence Query Language)

REFERENCIAS BIBLIOGRÁFICAS

1. H2 Database Documentation: <https://www.h2database.com/>
2. Spring Data JPA Documentation: <https://docs.spring.io/spring-data/jpa/docs/current/reference/html/>
3. Jakarta Persistence Specification: <https://jakarta.ee/specifications/persistence/>
4. Bauer, C., King, G. & Gregory, G. (2015). *Java Persistence with Hibernate*. Manning Publications.
5. Fowler, M. (2002). *Patterns of Enterprise Application Architecture*. Addison-Wesley.
6. Vernon, V. (2013). *Implementing Domain-Driven Design*. Addison-Wesley.